

COGNOME:

NOME:

MATRICOLA:

Appello di Algoritmi e Strutture Dati (9CFU), 30 Gennaio 2024.

 Parte I - Usare voto prova parziale - SI() NO()

Esercizio 1 [3 punti]

Quale delle seguenti sequenze di funzioni è tale che ogni funzione è O della successiva?

(A) $\log \log n, n^2, \sqrt{n} \cdot (\log n)^{2024}, 2^{\log n^{2024}}$

(B) $\sqrt{n}, \binom{n}{2024}, n^{\sqrt{n}}, n^{2024} \cdot \binom{n}{2024}$

(C) $2^{(\log n)^{2024}}, n^{\sqrt{\log n}}, 2^{\sqrt{n} \log n}, n^{\log n}$

~~(D)~~ Nessuna delle altre

Esercizio 2 [4 punti]

Indicare la complessità temporale nel caso peggiore dell'algoritmo descritto di seguito. L'input consiste di due array di interi, X e A , di dimensione comune n .

Algorithm PREFIX(X, A, n)

1: **if** $n = 1$ **then** $\longrightarrow \Theta(1)$ 2: $A[0] = X[0]$ 3: **return** $A[0]$ 4: **end if**5: $tot = \text{PREFIX}(X, A, n - 1) \longrightarrow \tau(n-1)$ 6: $tot = tot + X[n - 1]$ 7: $A[n - 1] = tot/n$ 8: **return** tot

$$\tau(n) = \tau(n-1) + \Theta(1)$$

$$\tau(n) = \Theta(n)$$

~~(A)~~ $\Theta(n)$

(B) $\Theta(\log n)$ (C) $\Theta(2^n)$

(D) Nessuna delle altre

Esercizio 3 [3 punti] L'algoritmo quick-sort è

~~(A)~~ Un algoritmo di ordinamento ricorsivo tramite partizionamento

(B) Un algoritmo per partizionare insiemi

(C) Un algoritmo di ordinamento che sfrutta gli alberi

(D) Un algoritmo per la creazione di partizioni

 Parte II

Esercizio 4 [3 punti]

Un coda di priorità è

~~(A)~~ Una struttura dati per mantenere in modo ordinato un insieme di elementi, ciascuno associato ad una priorità

(B) Una struttura dati per la gestione delle code negli uffici postali

(C) Una struttura dati in cui il primo elemento ad essere inserito è sempre il primo ad essere estratto

(D) Un algoritmo di ordinamento per dati numerici

Esercizio 5 [4 punti]

Sia $G = (V, E)$ un grafo *diretto* con $|E| \geq 1$. Supponiamo di effettuare una visita in profondità di G e sia G_π la foresta DFS prodotta. Indicare tutte e sole le affermazioni vere.

- ☒ (A) Se esiste un cammino da u a v in G_π , allora $u.d \leq v.d$.
- (B) Se $u.d < v.d$ ed esiste un cammino da u a v in G , allora esiste un cammino da u a v in G_π .
- (C) Le componenti fortemente connesse di G sono gli alberi di G_π .
- ☒ (D) L'insieme degli archi d'albero di G non è vuoto.

Esercizio 6 [3 punti]

L'operazione di Union per insiemi disgiunti

- (A) Unisce due insiemi dinamici mantenendo i due rappresentati originari
- ☒ (B) Unisce due insiemi dinamici al fine di creare uno con un solo rappresentante
- (C) Dati n elementi li unisce ricorsivamente in un solo insieme
- (D) Unisce due elementi dello stesso insieme al fine di trovare un rappresentante

SBARRAMENTO - 12 punti sugli esercizi precedenti

Esercizio 7 [12 punti]

1. Si consideri l'algoritmo DFS. Se ne descriva brevemente l'idea, si forniscano pseudo codice e tempo di esecuzione nel caso peggiore, e si indichino almeno tre problemi risolvibili applicando DFS. **[8 punti]**
2. Sia $G = (V, E)$ un grafo diretto costruito da un certo studente nel modo seguente. L'insieme dei vertici è $V = I \cup C$, dove ogni vertice in I corrisponde ad un insegnamento erogato dall'Università di Parma mentre ogni vertice in C corrisponde ad un corso di studio dell'Università di Parma. Il grafo G ha un arco (u, v) se e solo se $u \in I$ e u è prerequisito per $v \in V$ (che può essere sia insegnamento che corso). A ciascun insegnamento $u \in I$ è assegnato un valore $w(u) \in \mathbb{R}$ (che può essere negativo), corrispondente al gradimento dello studente per l'insegnamento u .

Supponendo che G non abbia cicli, fornire un algoritmo efficiente che trovi un corso maggiormente gradito, cioè un vertice $c \in C$ che massimizzi la quantità $\sum_{u \rightsquigarrow c} w(u)$, dove la somma è effettuata su tutti i vertici u per i quali esiste un cammino da u a c . **[4 punti]**

Esercizio 1 [3 punti]

Quale delle seguenti sequenze di funzioni è tale che ogni funzione è O della successiva?

(A) $\log \log n$, n^2 , $\sqrt{n} \cdot (\log n)^{2024}$, $2^{\log n^{2024}}$

(B) $\sqrt{n}$, $\binom{n}{2024}$, $n^{\sqrt{n}}$, $n^{2024} \cdot \binom{n}{2024}$

(C) $2^{(\log n)^{2024}}$, $n^{\sqrt{\log n}}$, $2^{\sqrt{n} \log n}$, $n^{\log n}$

(D) Nessuna delle altre

A- $\log \log n \leq n^2 > \sqrt{n}$

• $\sqrt{n} = n^{1/2}$

B- $\sqrt{n} \leq \binom{n}{2024} \leq n^{\sqrt{n}} > n^{2024}$

• $\sqrt{n} = n^{1/2}$

• $\binom{n}{2024} = n^{2024}$

C- $2^{(\log n)^{2024}} > n^{\sqrt{\log n}}$

• $2^{(\log n)^{2024}} = 2^{(\log n)(\log n)^{2023}} = n^{(\log n)^{2023}}$

• $n^{\sqrt{\log n}} = n^{(\log n)^{1/2}}$

1. Si consideri l'algoritmo DFS. Se ne descriva brevemente l'idea, si forniscano pseudo codice e tempo di esecuzione nel caso peggiore, e si indichino almeno tre problemi risolvibili applicando DFS. [8 punti]

DFS: è un algoritmo di esplorazione sistematica che visita i vertici di un grafo andando il più in profondità possibile prima di effettuare il passo di ritorno.

L'idea si basa sull'esplorazione dei vicini e sulla colorazione, utilizza 3 colori:

- bianco: nodo non ancora scoperto.
- grigio: nodo scoperto ma la sua lista di adiacenza non ancora completata.
- nero: nodo interamente visitato.

- L'algoritmo mantiene inoltre 2 tempi per ogni nodo u : $u.d \rightarrow$ tempo di scoperta quando diventa grigio e $u.f \rightarrow$ tempo di fine visita e diventa nero
- La DFS esplora un ramo fino a quando non incontra nodi già scoperti, dopodiché torna indietro se al termine rimangono nodi bianchi, l'algoritmo seleziona una nuova sorgente e riparte

DFS(G):

```
for each vertex  $u \in G.V$ 
     $u.Color = White$ 
     $u.\pi = NIL$ 

time = 0
```

```
for each vertex  $u \in G.V$ 
    if  $u.Color == White$ 
        DFS-VISIT(G, u)
```

tempo tot. = $\Theta(|V| + |E|) = \Theta(n+m)$

DFS-VISIT(G, u)

```
time += 1
 $u.d = time$ 
 $u.Color = Gray$ 
```

```
for each  $v \in G.Adj[u]$ 
    if  $v.Color == White$ 
         $v.\pi = u$ 
        DFS-VISIT(G, v)
```

$u.Color = Black$

time += 1

$u.f = time$

3 Problemi:

1. ordinamento topologico: applicabile su grafi diretti aciclici DAG. Si esegue DFS, e si ordinano i nodi in modo decrescente rispetto al loro tempo di fine visita $u.f$.
2. Controllo dei cicli: un grafo contiene almeno un ciclo se e solo se durante la DFS viene scoperto almeno un arco all'indietro (arco da un nodo corrente a uno grigio).
3. Calcolo delle SCC: si utilizza DFS 2 volte, la seconda sul grafo trasposto G^T , selezionando i nodi in ordine decrescente rispetto ai tempi $u.f$ calcolati nella prima.

2. Sia $G = (V, E)$ un grafo diretto costruito da un certo studente nel modo seguente. L'insieme dei vertici è $V = I \cup C$, dove ogni vertice in I corrisponde ad un insegnamento erogato dall'Università di Parma mentre ogni vertice in C corrisponde ad un corso di studio dell'Università di Parma. Il grafo G ha un arco (u, v) se e solo se $u \in I$ e u è prerequisito per $v \in V$ (che può essere sia insegnamento che corso). A ciascun insegnamento $u \in I$ è assegnato un valore $w(u) \in \mathbb{R}$ (che può essere negativo), corrispondente al gradimento dello studente per l'insegnamento u .

Supponendo che G non abbia cicli, fornire un algoritmo efficiente che trovi un corso maggiormente gradito, cioè un vertice $c \in C$ che massimizzi la quantità $\sum_{u \rightsquigarrow c} w(u)$, dove la somma è effettuata su tutti i vertici u per i quali esiste un cammino da u a c . [4 punti]

- Supponendo che G non abbia cicli $\longrightarrow$ è un DAG.
- Per calcolare la somma dei gradimenti, sfruttiamo la programmazione dinamica su DAG, combinata con l'ordinamento topologico.
- Sfruttando l'ordinamento topologico, possiamo elaborare i nodi in una sequenza lineare, t.c. per ogni arco (u, v) , il nodo u precede sempre il nodo v .
- Quindi: quando esaminiamo un vertice u , abbiamo già accumulato e finalizzato tutto il gradimento proveniente dai suoi predecessori, permettendoci di propagarlo in avanti verso i suoi vicini in tempo lineare.

Trova_Corso_ottimo(G, W)

// Inizializzazione del gradimento

Per ogni vertice $v \in V$ do

se $v \in I$ allora $\text{grad}[v] = w(v)$

altrimenti: $\text{grad}[v] = 0$

) $OC(V)$

// ordinamento topologico

$O = \text{ORDINAMENTO_TOPOLOGICO}(G)$

$\rightarrow OC(V + |E|)$

// Propagazione del gradimento in avanti:

Per ciascun vertice $u \in O$ (da sx a dx) do

Per ciascun vertice $v \in \text{Adj}[u]$ do

$\text{grad}[v] = \text{grad}[v] + \text{grad}[u]$

) $OC(V + |E|)$

// Ricerca del massimo tra i corsi in C

$\text{max_grad} = -\infty$

$\text{corso} = \text{NIL}$

Per ciascun vertice $c \in C$ do

se $\text{grad}[c] > \text{max_grad}$ allora

$\text{max_grad} = \text{grad}[c]$

$\text{corso} = c$

) $OC(V)$

return corso

Totale = $OC(V + |E|)$